



Training Python-Django - 1

Base Django

Summary: After Python, Django!

Version: 1.3

Contents

I	Preamble	2
II	General rules	3
III	Today's Specific Rules	4
IV	Exercise 00	5
V	Exercise 01	6
VI	Exercise 02	8
VII	Exercise 03	10
VIII	Submission and Peer Evaluation	12

Chapter I

Preamble

Here is a list of monsters you may encounter wandering a dungeon in the land of Fangh:

- any kind of undead.
- giant spiders.
- orcs and goblins.
- trolls in the underground.
- sorcerers.
- cursed warriors.
- giant rats.
- an oil bottle.
- toilet paper.
- two sponges.
- raviolis.

Chapter II

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a 0 during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.



For obvious security reasons, any credentials, API keys, env variables etc... must be saved locally in a `.env` file and ignored by git. Publicly stored credentials will lead you directly to a failure of the project.

Chapter III

Today's Specific Rules

- All paths relative to an application must be defined in a `urls.py` file located in that application's folder.
- Any form (a class derived from `django.forms.Form`) must be located in the `forms.py` file of the application it is related to.
- Each displayed page must be properly formatted (a doctype, pairs of `html` tags, `body`, `head` must be included), with proper management of special characters and no strange display.
- Today, you will use Django's default development server provided with the `manage.py` utility.
- Only the specifically requested URLs must return a page without any error. Hence, if just `/ex00` is requested, `/ex00foo` must return a 404 error.
- Requested URLs must work with or without a trailing slash. Hence, if `/ex00` is requested, `/ex00` and `/ex00/` both must work.

Chapter IV

Exercise 00

	Exercise 00
Exercise 00: First Static Page	
Turn-in directory: <code>ex00/</code>	
Files to turn in: <code>requirements.txt</code> , files and folders of your project and application	
Allowed functions: All Django functionalities.	

With this exercise, you will create your first static page with Django.

Create a `virtualenv` with `python3`, install Django, create a `requirements.txt` file containing the project's dependencies (what is installed in the `virtualenv` with `pip`) at the root of the repository.

Start a project named `d05`.

Start an application named `ex00`.

Create a page that will be accessible at the following URL: `/ex00` on your website. On this page, gather all the information about the `Markdown` syntax and give the page the title "Ex00: Markdown Cheatsheet".

The template used must be named `index.html`.

Chapter V

Exercise 01

	Exercise 01
Exercise 01: A Few More Pages	
Turn-in directory: <code>ex01/</code>	
Files to turn in: <code>requirements.txt</code> , files and folders of your project and application.	
Allowed functions: All Django functionalities.	

Create the following pages in a second application named `ex01`:

- **Title:** "Ex01: Django, Web Framework."
URL: `/ex01/django`.
Description: On this page, briefly introduce Django and its history.
- **Title:** "Ex01: Display Process of a Static Page."
URL: `/ex01/display`.
Description: On this page, describe the process leading to the display of a static web page from a simple template through a view, from the request to the response.
- **Title:** "Ex01: Template Engine."
URL: `/ex01/templates`.
Description: On this page, describe how Django's default template engine works, as well as the functioning of:
 - Blocks.
 - `for ... in` loops.
 - `if` control structures.
 - The display of context variables.

The *Don't Repeat Yourself* principle is part of Django's philosophy. To respect this principle, create the required pages using a base template named `base.html`.

The `base.html` template must include:

- A content block in the `body`.
- A `style` block in the `head`.
- A `title` block in the `head`.

Also create a `nav.html` template containing a navigation bar listing links to each page of the exercise.

All the pages of this exercise must be based on the `base.html` template. The `nav.html` template must be included in each page.

Also create 2 files: `style1.css` and `style2.css`. The first one will define the text color as *blue*, while the second one will be *red*. You will have to use `style1.css` on every page except the "Template Engine" page, where you will use `style2.css`.

To respect the DRY principle and ensure clean CSS management, the `style` block in `base.html` should contain a `<link>` to `style1.css`. Then, in `ex01-template.html`, you must override this block to include `style2.css` instead.



You must use each stylesheet only once in all your templates for this exercise.



You will have to use `collectstatic` during the evaluation.

Chapter VI

Exercise 02

	Exercise 02
Exercise 02: First Form	
Turn-in directory: <code>ex02/</code>	
Files to turn in: <code>requirements.txt</code> , files and folders of your project and application.	
Allowed functions: All Django functionalities.	

Create a page accessible at the URL `/ex02` in a new `ex02` application.

This page will contain 2 parts:

- A form with a text field and a `Submit` button.
- A section containing the input history.

Here are the rules regarding the form:

- You must NOT hard-code the form's field. You must use the `django.forms.Form` class provided by Django to create the form. You will pass it as `context` to the template's render function.

Here are the rules for the history:

- The input history will start empty.
- For each text submission through the form, you will have to:
 - Append the input and its timestamp to a **logs** file. If the file doesn't exist, it must be automatically created. This file must be created in the folder of the **ex02** application.
 - Add the input to the page's history along with the submission date and time.
- The path to the **logs** file (including the file name) must be defined in the project's **settings.py** file. Hence, each submitted entry will be included in the page's history and in the **logs** file, preceded by its timestamp.
- The data must also be persistent. If the development server is restarted for any reason, the data must not be lost.
As a rule, if you redisplay the page, the data must still be displayed as long as they are saved.

Chapter VII

Exercise 03

	Exercise 03
Exercise 03: Fifty Shades of Bic	
Turn-in directory: <i>ex03/</i>	
Files to turn in: <code>requirements.txt</code> , files and folders of your project and application.	
Allowed functions: All Django functionalities.	

Create a final application named `ex03`.

Display a page containing a table with 4 columns and 51 rows (one row will be dedicated to the column names).

You will access this page at the URL `/ex03`.

Each table column will have a different color: *black*, *red*, *blue*, and *green*.

The table cells must have the following specifications:

Height: 40 pixels.

Width: 80 pixels.

Background color: a shade matching the column color.

The table must display the shades of each color so as to obtain a 50-row gradient.

You must observe the following instructions:

- Your template must NOT include hard-coded colors. The different shades must be generated in a view and must all be different.
- A total of 4 pairs of `<td></td>` tags are authorized in your templates.
- A total of 4 pairs of `<th></th>` tags are authorized in your templates.
- A table must be formatted as follows:
 - One pair of `<th></th>` tags per cell for the column names.
 - One pair of `<tr></tr>` tags per color-shade row.
 - One pair of `<td></td>` tags per color-shade cell.

Chapter VIII

Submission and Peer Evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your folders and files to ensure they are correct.



The evaluation process will take place on the computer of the evaluated group.